



OUTLIERS

agent-flow — Watch Your Agents Hand Work to Each Other, Live

A free screen that draws every Claude Code session and subagent as it works, installed
the safe way

PIECE 1 OF 4 • THE AGENT WORKSPACE

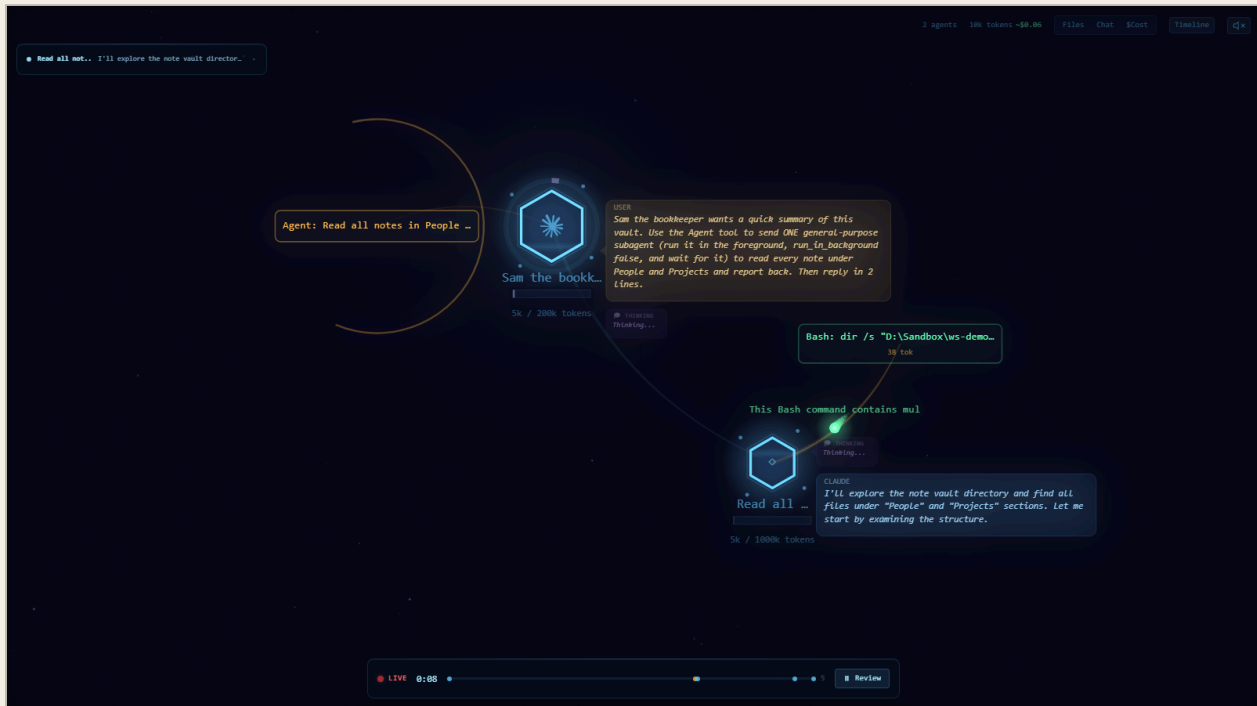
DOWNLOAD

<https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow>

EST. MMXXVI • OUTLIERS ACCELERATOR

What it is

agent-flow is a free web page that runs on your own computer. Open it in your browser at <http://127.0.0.1:3001> while Claude Code is working and you see a dark screen with a glowing hexagon. That hexagon is your Claude Code session. When the session hands a job to a subagent (a second Claude that does 1 task and reports back), a second hexagon appears and a line is drawn between them. Every tool call, such as reading a file or running a command, pops up as a small card beside the agent that made it.



A CLAUDE CODE SESSION (TOP) HAS JUST HANDED A JOB TO A SUBAGENT (BOTTOM RIGHT). MADE-UP DEMO VAULT, TAKEN 2026-09-22.

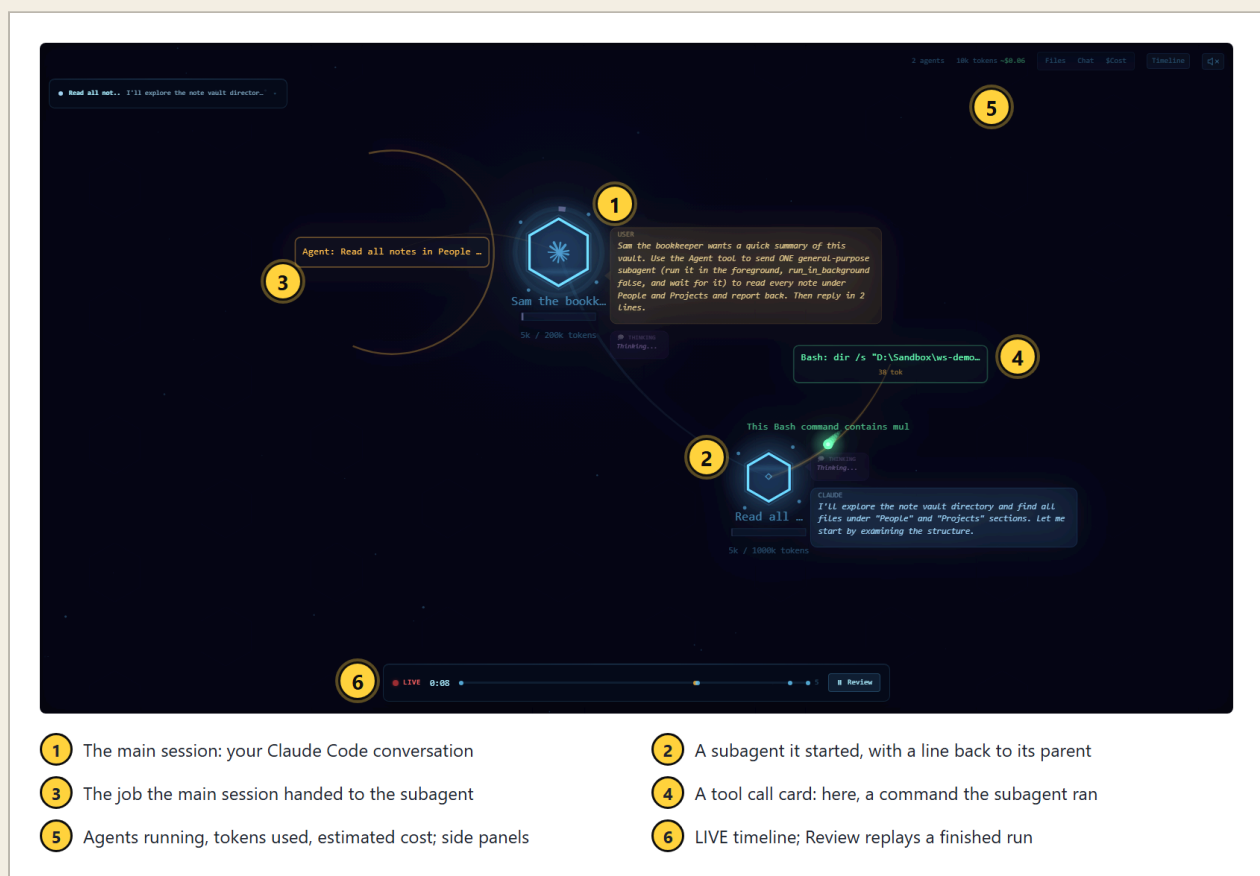
Along the top of the page are tabs, 1 per Claude Code session running on your computer. Each tab is titled with the first words of that session's opening message. Top right shows how many agents are running, an estimate of tokens used and an estimated cost. Along the bottom is a timeline with a LIVE marker and a Review button that replays what happened. There is also a sound button in the top right corner, which mutes the page's small sound effects.

agent-flow was written by another developer and published as open source under the Apache 2.0 licence (github.com/patoles/agent-flow; npm package `agent-flow-app`, version 0.9.1 checked on 2026-09-22). It also comes as a VS Code extension. Our download does not contain agent-flow. It contains an installer, a starter, 2 checks and a safety layer that set agent-flow up safely, because the stock install has faults on Windows that cost us weeks, and 1 fault on every computer that can wipe your Claude Code settings. All of them are explained below.

Why you would want it

When you run agents, most of the work is invisible. You type a request, the terminal scrolls, an answer arrives. You do not see which subagents were called, in what order, or which files they opened. That matters in 3 situations:

- **Building an agent.** Its instructions say it calls 3 specialists. Does it? On this screen you see every subagent it really starts and every file it really reads.
- **Something is slow or expensive.** You see the moment a session fans out into 5 subagents, or reads the same file 4 times.
- **Showing your work.** A client or a video viewer understands "the agents handed the job along" in 5 seconds when they watch it happen.

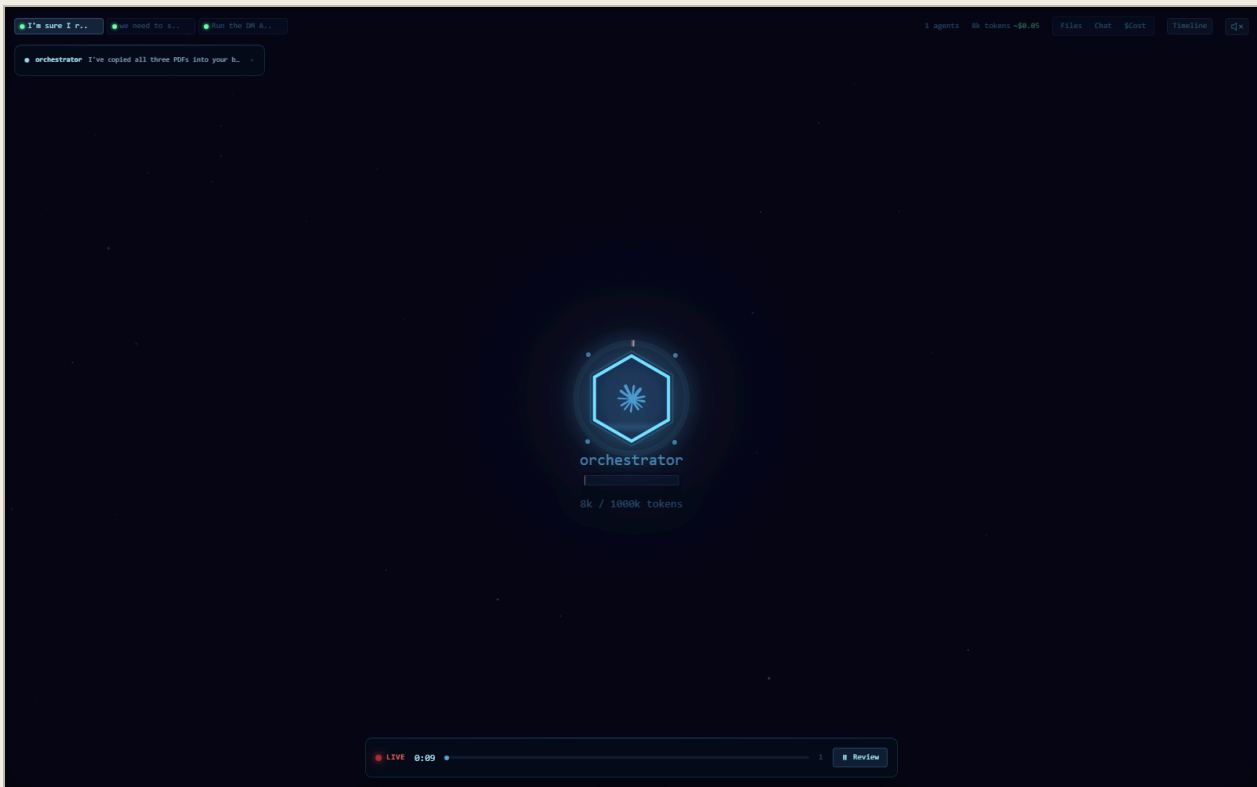


WHAT YOU CAN READ OFF THE SCREEN. THE NUMBERS ARE OURS, DRAWN ON A MADE-UP DEMO RUN.

It shows what happened. It does not change anything your agents do.

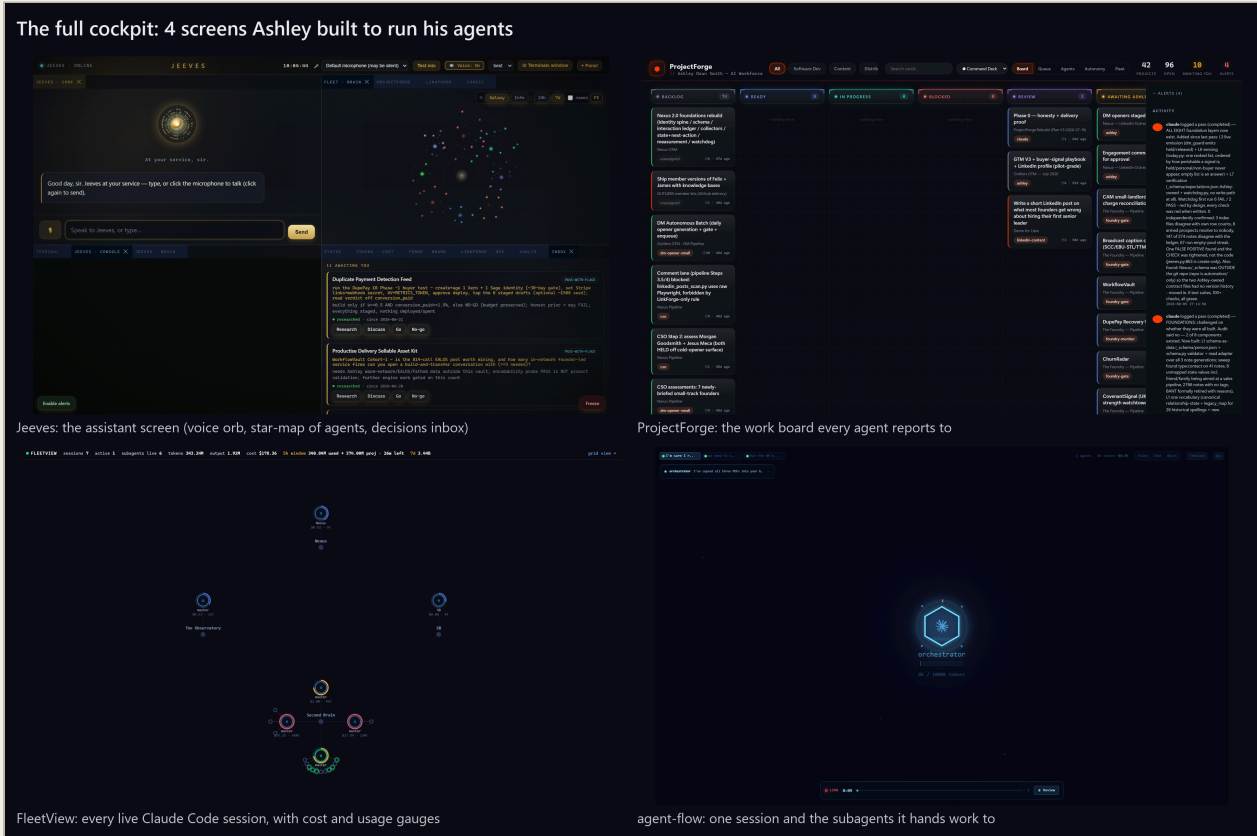
How we built it

This is how Ashley set agent-flow up on his own Windows PC, what broke, and what we kept. Every date and count below was read from his files, settings backups and logs on 2026-09-22.



ASHLEY'S OWN AGENT-FLOW, RUN ONCE HIDDEN ON 2026-09-22 FOR THIS GUIDE. THE 3 TABS ALONG THE TOP ARE HIS REAL CLAUDE CODE SESSIONS, TITLED WITH THE FIRST WORDS OF EACH PROMPT.

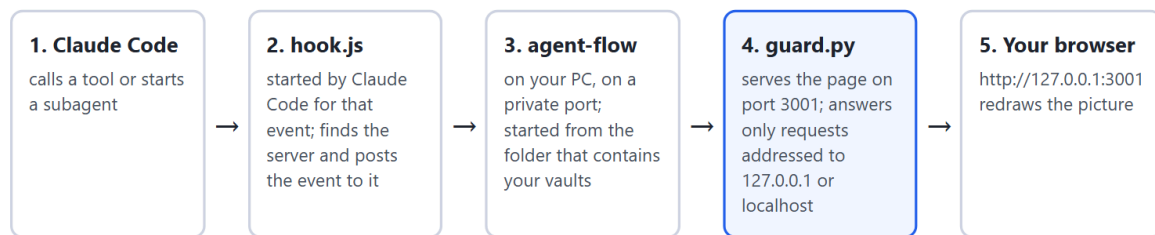
agent-flow is 1 of 4 screens Ashley built around his agents. The other 3 are the other pieces of this series.



ASHLEY'S REAL SCREENS SIDE BY SIDE, 2026-09-22: JEEVES (HIS ASSISTANT), PROJECTFORGE (THE WORK BOARD), FLEETVIEW (EVERY SESSION WITH COSTS) AND AGENT-FLOW (BOTTOM RIGHT).

First, how it works, because every fault below follows from it. Claude Code lets you register a **hook**: a command it runs every time a given event happens, such as "a tool is about to be used" or "a subagent has started". agent-flow registers a small script, `hook.js`, on 9 events. Each time it fires, Claude Code runs `hook.js` and hands it a description of the event. The script looks in the folder `<your home>/ .claude/agent-flow/` for small files that say where a running agent-flow server is listening, and sends the event there. The server updates its picture and your browser redraws.

How 1 event travels from Claude Code to your screen



Steps 1 to 3 are agent-flow's own design. Step 4 is ours, added on 2026-09-22: without it, a website could read the live stream of your conversation. Nothing on this path leaves your PC.

HOW 1 EVENT TRAVELS. STEP 4 IS OUR ADDITION FROM 2026-09-22, EXPLAINED FURTHER DOWN.

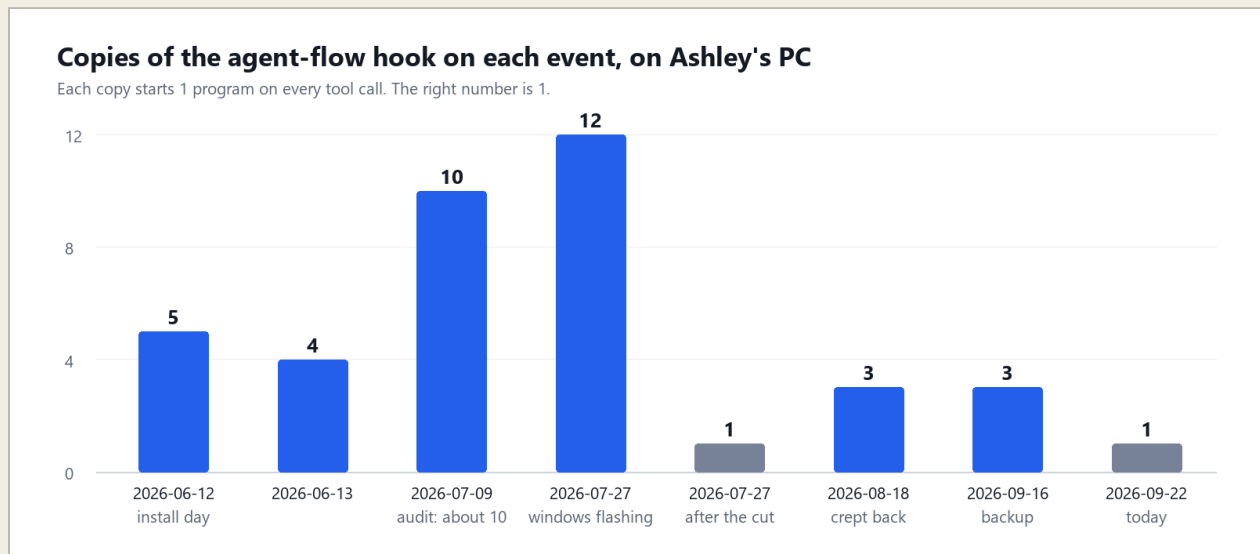
2026-06-12: INSTALL DAY

- Installed with `npx -y agent-flow-app`. (`npx` comes with Node.js; it downloads a Node.js program and runs it in 1 go.) On its first run the package wrote `hook.js` and added it to 9 events in Claude Code's `settings.json` file: `SessionStart`, `PreToolUse`, `PostToolUse`, `PostToolUseFailure`, `SubagentStart`, `SubagentStop`, `Notification`, `Stop` and `SessionEnd`.
- Windows fix 1, the launch folder.** The server remembers the folder it was started from, and only accepts events from Claude Code sessions running inside that folder. Started from the wrong place, it shows nothing. So it was set to start from the Documents folder, the parent of every vault.
- Start at login, hidden.** A file called `agent-flow.vbs` went into the Windows Startup folder. It runs `npx -y agent-flow-app --no-open` with the window hidden.
- The merged view, tried and reverted.** Ashley wanted every session on 1 screen instead of 1 tab each. A patch to agent-flow's server merged all sessions into 1 stream. The data looked right, and it was reported as done without anyone looking at the screen. Ashley's reply: "that hasn't worked - It's not working at all." agent-flow's drawing code supports exactly 1 main hexagon per screen, so 6 main hexagons stacked on the same spot and their labels garbled. It was put back to the stock package. The lesson kept from it: check any change to a screen with a screenshot, never by reading the data behind it.
- The 1-screen wish went to a different tool, FleetView, which is piece 2 of this series.
- A backup of `settings.json` taken at 17:00 that day already had **5 copies** of the agent-flow hook on every event. The next day's backup had 4.

WINDOWS FIX 2: STALE REGISTRATION FILES (DATE NOT RECORDED)

Each time the server starts it writes a file named `<code>-<process number>.json` into `<home>/ .claude/agent-flow/` , saying which port it listens on and which folder it watches. (A port is the number a program on your computer answers on; the page uses 3001.) The hook is meant to delete files for servers that have stopped. On Windows its "is this server still alive?" check is written to always answer yes. Worse, when several files match a session's folder, the hook sends only to the server watching the **narrowest** folder. So an old file for a narrower folder swallows every event while the real server sits idle, and the screen just stays empty. The fix was to delete the old files and keep the one whose server is running. 13 such files were still sitting in that folder on 2026-09-22, dated 2026-06-12 to 2026-08-06.

2026-07-09 TO 2026-09-16: THE HOOK COPIES PILE UP



COPIES OF THE AGENT-FLOW HOOK ON EACH EVENT, FROM SETTINGS BACKUPS AND AUDITS.

- **2026-07-09:** an audit found the hook registered about 10 times on every event.
- **2026-07-19:** `npx` pulled the newer agent-flow 0.9.1.
- **2026-07-27:** Ashley: "Instances keep opening on my PC." The settings file had 112 hook entries in total; agent-flow's was on every event 12 times. Each tool call started about 27 small console programs, and each flashed a window on screen. Everything was cut to 1 copy per event and command: 112 entries down to 13.
- **2026-08-18:** it had crept back to 3 per event.
- **2026-09-16:** a backup shows 3 per event. On 2026-09-22 the file shows 1 per event. No written record of that second cleanup was found.

At the time, why the copies piled up was not recorded. While building this kit on 2026-09-22 we read agent-flow 0.9.1's own set-up code and found a cause. Before adding its hook, agent-flow checks whether it is already there by searching your settings for the text `agent-flow/hook.js` , with a forward slash. On Windows the path it writes uses backslashes: `agent-flow\hook.js` . The search never matches its own entry, so **every time the server starts, it adds another copy to all 9 events**. We tested it in a throwaway folder: 3 starts gave 3 copies per event. A server that starts at every logon, plus manual restarts during fixes, matches every rise in the chart. We cannot prove it was the only cause, because nobody logged each start, but it explains every rise above.

Our fix changes only the direction of the slashes: the installer writes the same hook with forward slashes (`C:/Users/<you>/.claude/agent-flow/hook.js`). Windows accepts forward slashes, and agent-flow now recognises its own entry and leaves your settings alone. Tested the same day: with our line in place, 2 more starts changed nothing.

THE USAGE-TRACKING FINDING

agent-flow 0.9.1 sends anonymous usage events (an install code, its version, your operating system, session length) to a server run by its author, unless you set `AGENT_FLOW_TELEMETRY=false` or `DO_NOT_TRACK=1` before starting it. The GitHub page summary says tracking is off by default; the code says on. Trust the code. On Ashley's PC, 4 such events were logged between 2026-07-19 and 2026-08-06, because his launcher set neither switch. Our launcher sets both.

2026-08-06: SWITCHED OFF

At 17:25 on 2026-08-06 the Startup entry for agent-flow was switched off in Windows' Startup apps list, in the same minute as the FleetView dashboard. Who did it and why was not written down. His failure log for 2026-09-22 says these screens were "stopped not deleted" when his work changed direction. The hooks were left in place, so every tool call still started 1 `node` program that found no server and quit. That is a small, silent cost, and our uninstall removes it.

2026-09-22: WHAT A SECURITY CHECK OF OUR OWN KIT FOUND

Before this guide went out, the kit was tested by someone trying to break it, using the real agent-flow package in a throwaway home folder. 3 faults were found and all 3 are fixed in the version you download.

- **agent-flow could wipe your Claude Code settings at logon.** agent-flow runs its own set-up every time it starts. If it cannot read `settings.json`, it throws the whole file away and writes a new one containing only its 9 hooks: your permission rules, your blocked commands, your model choice and every other hook, gone, with no backup. 2 ordinary slips make the file unreadable to it: a comma after the last item (easy to leave after a hand edit), and an invisible marker at the very start of the file called a byte-order mark, which Windows PowerShell and some editors add when they save. With a logon launcher, this happened silently the next time you logged in. **Our fix:** `start.py` now checks the file before every start and does not start agent-flow if the file is unsafe; it says why on screen and in `logs/start.log`. As a second layer, a new file, `guard.py`, keeps an exact copy of your settings before agent-flow starts, watches the file for the first 60 seconds, and puts your copy back if agent-flow changed it. Tested with the real package: with a trailing comma or a byte-order mark, the file stays byte-for-byte the same.
- **Stop could close an unrelated program.** `start.py --stop` used to stop whatever program had the process number saved when agent-flow last started. After a restart, Windows can give that number to any program, including Claude Code or your browser. **Our fix:** the saved record now includes the moment the process started, and stop only acts when both match.
- **Other websites could read the page.** agent-flow's page streams your live conversation: your prompts, Claude's replies, file names and commands. It answers every request that reaches it, whatever web address the browser thinks it is using. A website can exploit that with a trick called DNS rebinding (a web address that first points at the website, then at your own computer) and read the stream while you browse. **Our fix:** agent-flow now runs on a private port and `guard.py` serves

the page on 3001, answering only requests addressed to `127.0.0.1`, `localhost` or `:::1`. Anything else is refused.

WARNING

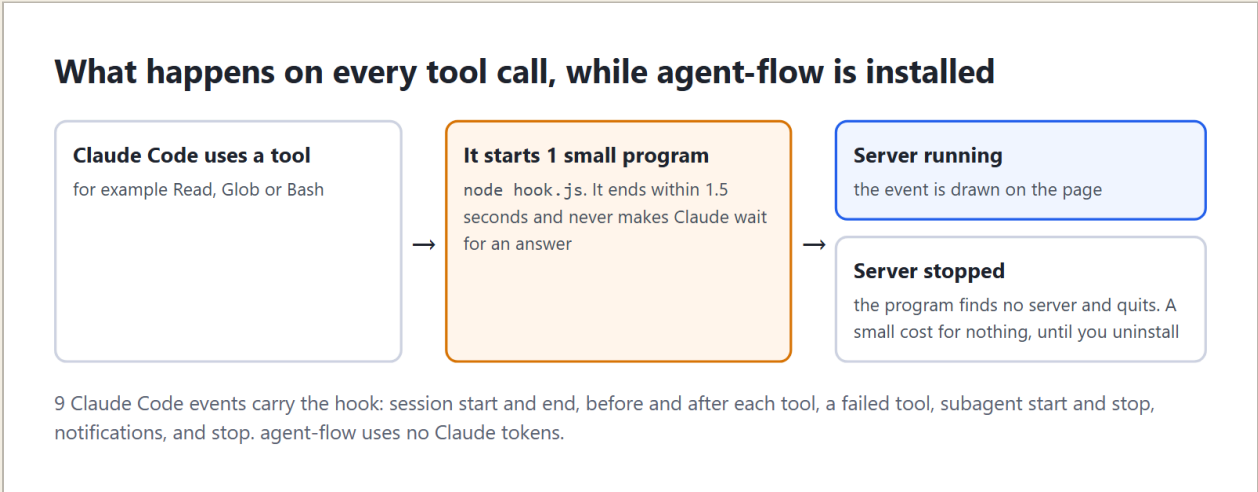
1 exposure remains, and it is agent-flow's, not ours to fix. The separate port that receives events from `hook.js` accepts them from any web page, so a website that guesses that port number (it changes at every start) could draw fake sessions and fake tool calls on your screen. It cannot read anything through that port. If you are not watching agent-flow, stop it with `python start.py --stop`. A real fix needs a change in agent-flow itself (github.com/patoles/agent-flow).

WHAT WE KEPT FOR YOU

- The stock agent-flow package, installed by you from npm. No patched copy.
- The forward-slash hook line, so copies never pile up.
- A hidden launcher that starts from the folder that contains your vaults, with usage tracking off.
- `cleanup.py` for the stale files, run automatically before every start.
- `check_hooks.py`, a count of every hook on every event.
- `guard.py`, the settings check and the page-address check from the security test.

Pros and cons

	PROS	CONS
Cost	Free and open source. Uses no Claude tokens itself.	Every tool call starts 1 small <code>node</code> program (it ends within 1.5 seconds and never makes Claude wait). It does this even when the server is off, until you uninstall.
Setup	About 1 minute with our installer, plus the first download.	Needs Node.js, which most people do not have yet.
What you see	Subagents appear the moment they start; every tool call is a card; Review replays a run.	1 session per tab. There is no single screen with every session (our attempt failed, see above).
Accuracy	Tool calls and subagents come straight from Claude Code's own events.	Token counts and costs are estimates. On Ashley's PC they disagreed with FleetView for the same session.
Privacy	The page only listens on your own computer, and our guard refuses requests addressed to anything but your own computer.	Tab titles show the first words of your prompts, so take care when screen-sharing. Usage tracking is on unless switched off (our launcher switches it off). Fake events can still be drawn by a website (see the warning above).
Safety	Our kit will not start agent-flow on a settings file it would wipe, and puts the file back if it changes anyway.	agent-flow's own set-up still runs at every start; our checks surround it, they do not remove it.
Windows	Our kit fixes the 2 known Windows faults.	Stale registration files still appear if the server is killed rather than stopped; <code>start.py</code> cleans them before each start.



THE COST OF HAVING THE HOOK INSTALLED: 1 SHORT-LIVED PROGRAM PER TOOL CALL, WHETHER OR NOT THE PAGE IS OPEN.

Before you start

YOU NEED	HOW TO CHECK	IF IT IS MISSING
Claude Code	<code>claude --version</code>	You have it from earlier sessions.
Python 3.8 or newer	<code>python --version</code> (Mac: <code>python3 --version</code>)	python.org
Git	<code>git --version</code>	git-scm.com
Node.js 18 or newer (20 recommended)	<code>node --version</code>	Windows: the LTS installer (long-term support, the stable version) from nodejs.org , or <code>winget install OpenJS.NodeJS.LTS</code> . Mac: nodejs.org or <code>brew install node</code> . Then open a NEW terminal, because a terminal opened before the install cannot find Node.js.
Port 3001 free (the number the page is served on)	Open http://127.0.0.1:3001 in a browser; it should fail to load	Install with <code>python install.py --port 3002</code> instead, then use 3002 wherever this guide says 3001.

Windows PowerShell - the 4 checks, as they printed on a test PC on 2026-09-22

```
> claude --version
2.1.278 (Claude Code)
> python --version
Python 3.13.14
> git --version
git version 2.54.0.windows.1
> node --version
v26.3.0
```

THE 4 CHECKS AS THEY PRINTED ON A TEST PC ON 2026-09-22. YOUR VERSION NUMBERS CAN BE HIGHER.

You also need to know where your second brain vault and your CRM vault live. The installer finds them for you in 1 of these ways: small pointer files called `.outliers-sb` and `.outliers-crm` in your home folder (our earlier kits left them), a folder with an `.obsidian` folder inside it (that is what makes a folder an Obsidian vault) up to 2 levels under your home or Documents folder, or, for the CRM, a vault whose name contains `crm`, `pipeline`, `sales` or `clients`, or a folder at `C:\Users\<you>\CRM`.

Install it

- 1. Open a terminal in your home folder (Windows: PowerShell; Mac: Terminal).
- 2. Run the clone and install line:

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow; cd outliers-ws-01-agent-flow; python install.py
```

On a Mac use `python3`. Keep the folder where it lands: the logon launcher points at it.

3. Answer 3 questions. Press Enter to accept the suggestion in square brackets.
 - Where is your second brain vault?
 - Where is your CRM vault?
 - Which folder should agent-flow watch? It suggests the folder that contains both vaults. It must contain every folder you run Claude Code in, or those sessions will not show.

If a question shows no suggestion, paste the full path of that vault (in File Explorer, click the address bar and copy it). No CRM vault yet? Press Enter to leave it blank.

4. Wait for the first run. The installer downloads agent-flow and starts it once, hidden, with a throwaway home folder, so it writes its own `hook.js` without touching your settings. It copies that script into your `.claude/agent-flow` folder and stops it. This can take up to 3 minutes on a slow connection. No login is needed.
5. Read the table. Every event should show `1` in the "after" column. If you had old copies, the "before" column shows how many, and a dated backup of your settings is named on screen. The logon launcher is `outliers-agent-flow.vbs` in your Startup folder; that is the name you see in Windows' Startup apps list.

```
What a first install prints (3 times Enter pressed)

> python install.py
agent-flow installer (Outliers workspace, piece 1)

Node.js 26.3.0 found.
Where is your second brain vault? [C:\Users\<you>\Documents\Second Brain]:
Where is your CRM vault? [C:\Users\<you>\Documents\Priya Shah CRM]:
Which folder should agent-flow watch? It must CONTAIN every vault you run Claude Code in [C:\Users\
<you>\Documents]:
Saved settings to C:\Users\<you>\outliers-ws-01-agent-flow\config.json
First run: downloading agent-flow-app@0.9.1 and letting it write its hook script (can take up to 180 seconds)...
Hook script present: C:\Users\<you>\.claude\agent-flow\hook.js

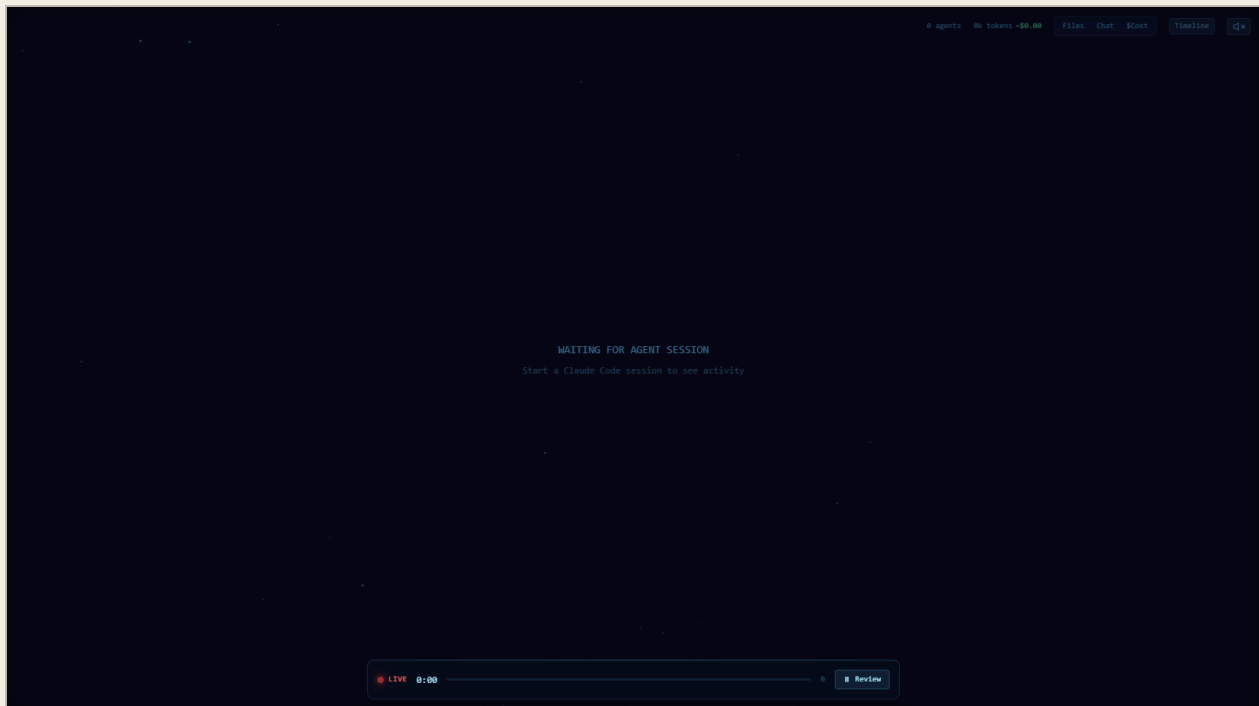
Claude Code settings: C:\Users\<you>\.claude\settings.json
Event      before  after
SessionStart      0      1
PreToolUse        0      1
PostToolUse       0      1
PostToolUseFailure 0      1
SubagentStart     0      1
SubagentStop      0      1
Notification      0      1
Stop              0      1
SessionEnd        0      1
Backup taken first: C:\Users\<you>\.claude\settings.json.bak-agent-flow-20260922-140220
Removed 0 extra copies; each event now has exactly 1.
The hook line now uses forward slashes, so agent-flow recognises it and will not add another copy each time it
starts.

Logon launcher written: C:\Users\<you>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\outliers-
agent-flow.vbs
Usage tracking: off (AGENT_FLOW_TELEMETRY=false and DO_NOT_TRACK=1 are set by the launcher).

Done. Start it now with: python start.py
Then open http://127.0.0.1:3001 and start a NEW Claude Code session inside the watch folder.
Check your hooks any time with: python check_hooks.py
```

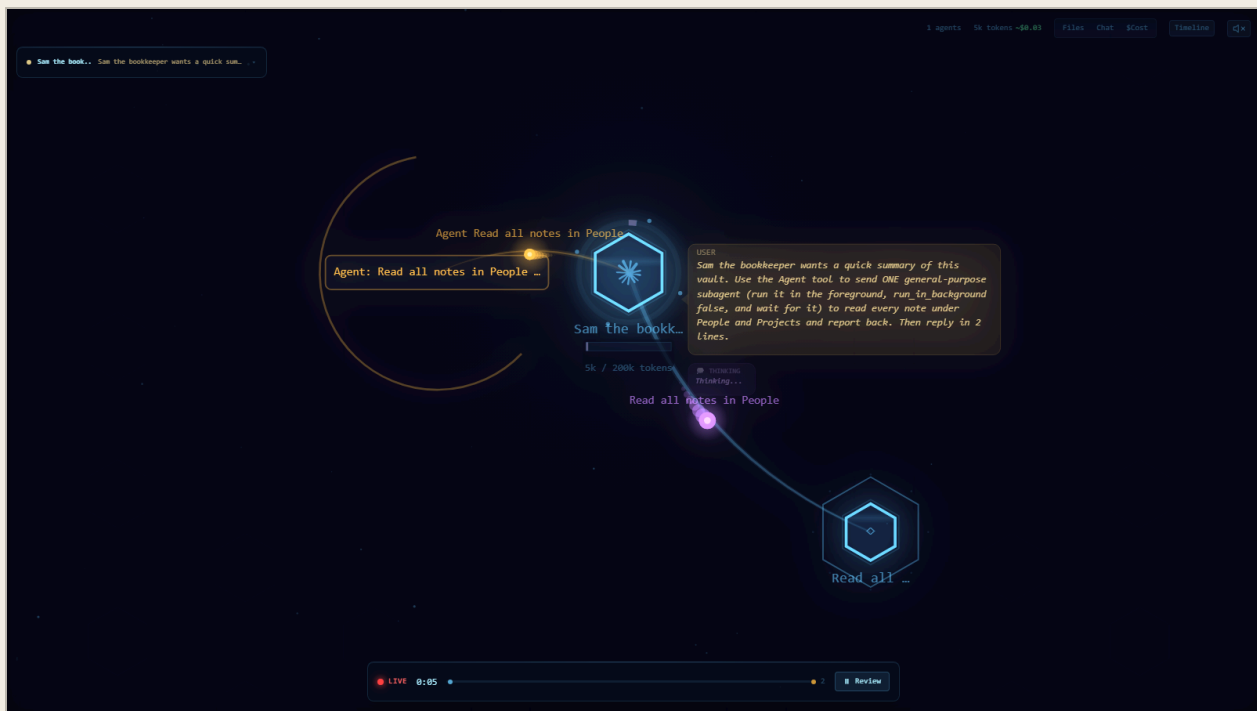
WHAT A SUCCESSFUL INSTALL PRINTS. PATHS SHORTENED TO C:\USERS\<YOU>; THE DEMO RAN IN A TEST FOLDER
ON 2026-09-22.

6. Start it now: `python start.py` . From now on it also starts by itself, hidden, each time you log in.
7. Open <http://127.0.0.1:3001>. You see "Waiting for agent session".



THE PAGE BEFORE ANY SESSION HAS STARTED.

8. Open a **new** Claude Code session inside one of your vaults. Sessions that were already open show nothing, because Claude Code reads hooks only when a session starts. Ask it for something that uses a subagent, for example: "Use a subagent to list the notes in my People folder, then summarise them in 2 lines." A hexagon appears, then a second hexagon with a line to it. Nothing after 30 seconds? Check the session was started AFTER step 6 and inside the watch folder shown by `python start.py --status` , then see When it goes wrong.



THE MAIN SESSION (TOP LEFT) STARTING A SUBAGENT. THE ORANGE LABEL IS THE JOB IT WAS GIVEN.

9. Run `python check_hooks.py` . It should end with `RESULT: OK` .

TIP

to remove everything later, run `python install.py --uninstall` . It stops the server, takes a backup, removes the 9 hook entries, and deletes the launcher. Your settings go back to how they were (byte for byte, when the first install's backup is still there). If you had no settings file before, a file containing only `{}` is left. The folder `.claude\agent-flow` stays; you may delete it by hand.

PIECE 1 • AGENT-FLOW

Using it day to day

Every command below runs inside the downloaded folder. In a new terminal, type `cd outliers-ws-01-agent-flow` first.

- **Leave the page open in a browser tab** while you work. Switch tabs at the top to follow a different session.
- **Press Review** on the bottom bar to replay a run that has finished.
- **The Files, Chat, Cost and Timeline buttons** top right open side panels: which files were touched, the conversation, the estimated cost, and a timeline of every call.
- **Check your hooks once a week:** `python check_hooks.py` . Anything other than 1 per event means something else has been writing to your settings.
- **If the page goes quiet:** run `python start.py --stop` , then `python start.py` , then open a new Claude Code session.
- **Stopping:** `python start.py --stop` says "Stopped agent-flow." or "agent-flow was not running. Nothing to stop." Check any time with `python start.py --status` .

- **If you move your vaults**, run `python install.py` again and give the new watch folder. The suggestion in square brackets is your OLD watch folder, so type the new one. If the server was running, the installer stops it and starts it again for the new folder.
- **Screen-sharing**: close the tab or pick a demo session first. Tab titles show the start of your prompts.
- **When you are not watching**, stop it. The fewer hours it runs, the fewer hours the page is there to be reached.

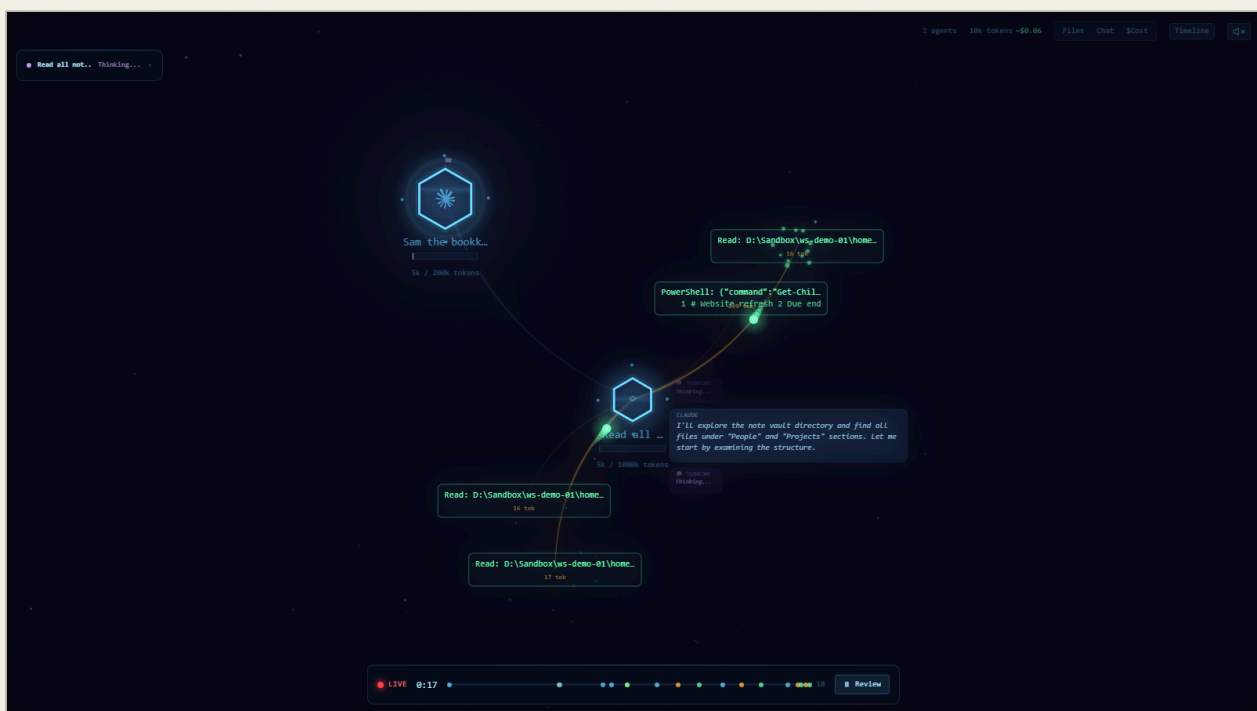
```

Checking, stopping, and stopping again

> python start.py --status
Running for C:\Users\<you>\Documents - http://127.0.0.1:3001
> python start.py --stop
Stopped agent-flow.
> python start.py --stop
agent-flow was not running. Nothing to stop.

```

CHECKING, STOPPING, AND STOPPING AGAIN, AS IT PRINTS.



A SUBAGENT AT WORK: EACH CARD IS 1 TOOL CALL IT MADE, HERE READING NOTES AND LISTING A FOLDER.

PIECE 1 • AGENT-FLOW

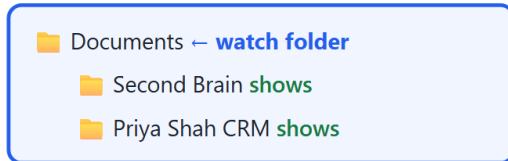
Fit it to your own AI system

Each idea below comes with a prompt you can paste into Claude Code, opened in the `outliers-ws-01-agent-flow` folder unless it says otherwise.

The watch folder must contain every vault you use

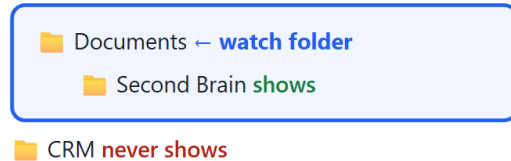
Works: both vaults inside Documents

📁 C:\Users\<you>



Fails: the CRM sits outside it

📁 C:\Users\<you>



Fix: watch C:\Users\<you> instead (idea 1 below).

IDEA 1 IN A PICTURE: THE WATCH FOLDER MUST CONTAIN EVERY VAULT WHOSE SESSIONS YOU WANT TO SEE.

1. **See your second brain AND your CRM in 1 place.** The server only hears sessions inside its watch folder. If your CRM lives at `C:\Users\<you>\CRM` and your second brain in `Documents`, the only folder containing both is your home folder.

```
Read config.json. Tell me whether my second brain vault and my CRM vault are both inside watch_folder. If not, run install.py again with --watch-folder set to the nearest folder that contains both, and show me the before and after.
```

2. **Watch the content engine write a wave.** Run your content engine from a folder inside the watch folder, open agent-flow, and start a wave. You will see which steps call subagents and which run as plain code.

```
My content engine lives at <path>. Check whether it is inside the watch_folder in config.json. If it is not, tell me the smallest change: move the folder, or widen the watch folder. Do not move anything without asking.
```

3. **Check an agent you are building really does what its instructions say.** Open the agent's file, list what it claims to call, run it with agent-flow open, and compare.

```
Read ~/.claude/agents/<agent-name>.md. List every subagent, tool and file it says it uses, as a checklist. I will run it now with agent-flow open and tell you what cards appeared. Then mark each checklist line as seen, not seen, or seen but not listed.
```

4. **Give every subagent a real name.** A subagent started without a registered name shows as an anonymous hexagon. Named agents make the picture readable.

```
Search my second brain's CLAUDE.md and my agent files for places that start a subagent with a generic type such as general-purpose. List each one and suggest which named agent from ~/.claude/agents/ should be used instead. Change nothing yet.
```

5. **Keep a permanent log.** agent-flow keeps no history once the server stops. Add a second, separate hook that appends each event to a dated file, then summarise a day into your second brain.

Add ONE new hook to my Claude Code settings.json on SubagentStart and SubagentStop that appends the event as 1 JSON line to ~/.claude/agent-events/YYYY-MM-DD.jsonl. Use a Python script run with pythonw.exe so no window opens. Take a dated backup of settings.json first, write it with a temp file and os.replace, and run python check_hooks.py afterwards to prove every event still has 1 copy of each command.

6. **Stop the empty spawns when the server is off.** If you only use agent-flow sometimes, every tool call still starts `node` for nothing. Remove the hooks when you stop, and put them back when you start.

Write stop_all.py and start_all.py in this folder. stop_all.py runs start.py --stop and then removes the agent-flow hooks using common.remove_agent_flow, with a backup. start_all.py puts the hooks back with common.install_agent_flow and runs start.py. Add tests using a temporary home folder, like the ones in tests/, and run them with python -m pytest -q.

7. **Run it only when you want it.** Skip the logon launcher and make a desktop shortcut instead. `--no-autostart` also removes a launcher you already have.

Run python install.py --no-autostart and check that outliers-agent-flow.vbs is no longer in my Startup folder. Then create a desktop shortcut called "Agent screen" that runs start.py with pythonw.exe (Windows) so no window appears, and a second one that runs start.py --stop.

8. **Capture footage for content.** Record the page during a real multi-agent run: it makes short video of "the agents working" for a post.

Write capture.py: open http://127.0.0.1:3001 in headless Playwright at 1600 by 900, take a screenshot every 2 seconds until I press Ctrl+C, and save them numbered into a folder called captures with today's date. No visible browser window.

9. **Weekly hook check without a window.** Run the duplicate check once a week, hidden, and write the result into your second brain's inbox only when something is wrong. `check_hooks.py` ends with exit code 1 when it finds a duplicate and 2 when settings.json cannot be read.

Create a Windows scheduled task that runs check_hooks.py once a week on Monday at 09:00 through pythonw.exe, hidden. If the exit code is 1 or 2, write a short note into <second brain>/Inbox/ named YYYY-MM-DD hook check.md with the output. Do not run it more often than weekly.

Every command and setting

outliers-ws-01-agent-flow: what is in the download

install.py	the interview: finds your vaults, installs the hook, adds the logon launcher
start.py	starts and stops agent-flow; checks settings.json first
guard.py	new: runs agent-flow, watches settings.json, guards the page address
check_hooks.py	counts every hook on every event; --fix removes repeats
cleanup.py	removes stale server registration files
common.py	shared code the 5 files above use
config.json	made by install.py: your vault paths, watch folder, port
logs/	made at run time: start.log (the kit's notes), agent-flow.log (agent-flow's own output)
tests/	python -m pytest -q; a made-up home folder, never your real one
guide/	this guide, as GUIDE.md and a PDF
README.md	a short summary and the useful commands
WHAT-I-STOLE.md	what we took from agent-flow, and its licence
LICENSE	MIT: use and change our files freely

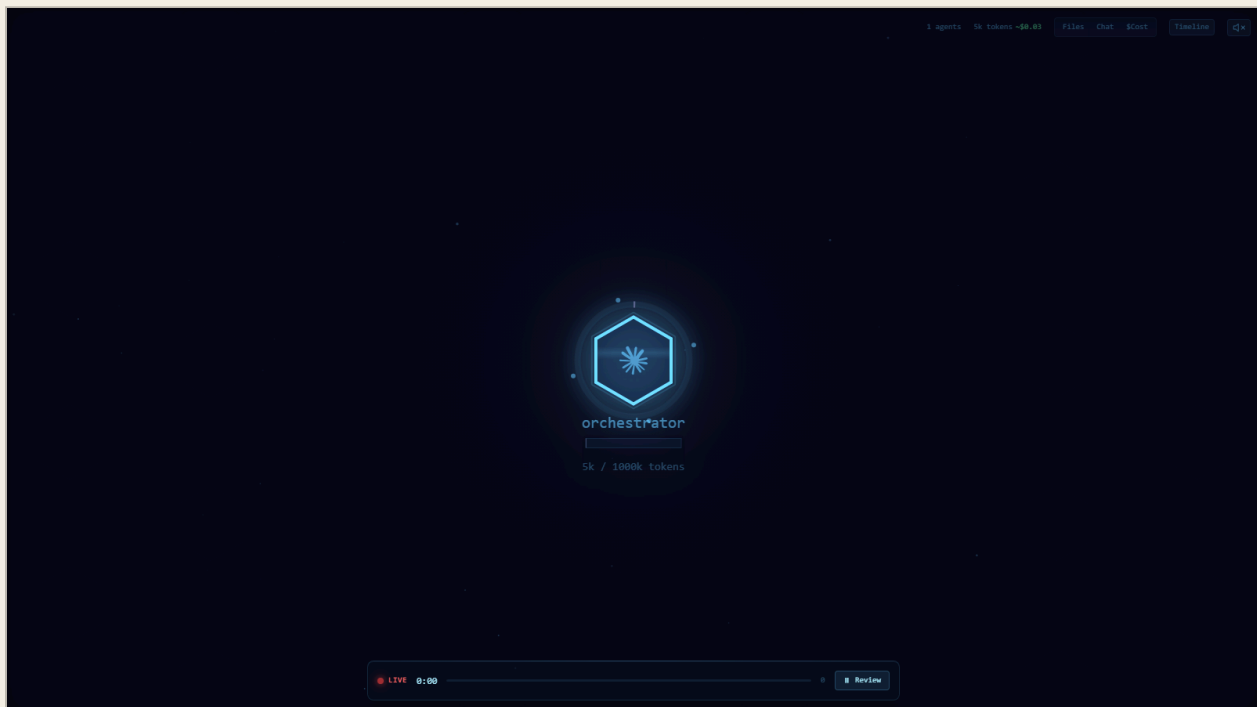
WHAT IS IN THE DOWNLOAD, AND WHAT EACH FILE DOES.

install.py (every option can be combined with the others)

OPTION	WHAT IT DOES
(none)	The interview: 3 questions, then the install.
<code>--yes</code>	Accepts every suggestion without asking.
<code>--second-brain <path></code> , <code>--crm <path></code>	Gives the vault paths instead of answering the questions.
<code>--watch-folder <path></code>	Sets the watch folder and skips the 3 questions. The installer warns you if a vault is not inside it.
<code>--port <number></code>	The page's port (default 3001). A running server is moved to the new port.
<code>--no-autostart</code>	No logon launcher; also removes a launcher you already have.
<code>--start-now</code>	Starts the server at the end, so you can skip step 6.
<code>--uninstall</code>	Stops the server, removes the 9 hooks (backup first) and the launcher.
<code>--package</code> , <code>--node-path</code> , <code>--wait</code> , <code>--startup-dir</code>	Advanced and for testing: which agent-flow version to run (default <code>agent-flow-app@0.9.1</code>), which <code>node</code> program goes into the hook, how many seconds to wait for the first download (default 180), and a different Startup folder.

start.py

OPTION	WHAT IT DOES
(none)	Checks settings.json, cleans stale files, starts agent-flow hidden, prints the address.
<code>--stop</code>	Stops the server this kit started.
<code>--status</code>	Says "Running" or "Not running", and if the last start was refused, why. Exit code 0 when running, 1 when not.
<code>--port</code> , <code>--watch-folder</code> , <code>--pack</code> <code>age</code> , <code>--wait</code>	Override <code>config.json</code> for this run only.
<code>--foreground</code> , <code>--quiet</code>	Used by the launchers: wait instead of returning (the Mac logon job), and print nothing (the Windows launcher).



THE PAGE SERVED THROUGH GUARD.PY FROM A MADE-UP SESSION, 2026-09-22: 1 AGENT, 5K TOKENS.

check_hooks.py: `--fix` keeps 1 copy of each repeated hook after a dated backup; `--settings <file>` checks a different settings file. Exit codes: 0 all well, 1 duplicates found, 2 the file cannot be read.

cleanup.py: `--dry-run` only reports what it would remove; `--quiet` prints nothing.

Files the kit makes while it runs

- `config.json` : your 2 vault paths, the watch folder, the agent-flow version, the port and whether the launcher is on. `config.example.json` shows the shape.
- `logs/start.log` : the kit's own notes, such as a refused start or a settings file put back.
- `logs/agent-flow.log` : agent-flow's own output. Read it when the server stops straight away.
- `logs/agent-flow.pid` : the number and start time of the running server, so `--stop` finds it and nothing else.

- Settings backups next to `settings.json` : `settings.json.bak-agent-flow-<date>` (before install), `.bak-agent-flow-uninstall-<date>` , `.bak-hookdedupe-<date>` (before `--fix`) and `.bak-agent-flow-undo-<date>` (agent-flow's rewritten version, kept when `guard.py` put yours back).
- The launcher: on Windows `outliers-agent-flow.vbs` in your Startup folder; on a Mac a login job, `~/Library/LaunchAgents/com.outliers.agent-flow.plist` , which runs at your next login (to start it now: `launchctl load` followed by that path).

Worth knowing

- If you set `CLAUDE_CONFIG_DIR` to keep Claude Code's settings somewhere else, the installer uses that folder. agent-flow itself always uses `<home>/.claude` .
- agent-flow also watches OpenAI Codex sessions unless `AGENT_FLOW_RUNTIME=claude` is set. It does no harm if you do not use Codex.
- The tests: `python -m pytest -q` . They use a made-up home folder and a stand-in for agent-flow. 4 more run against the real package if you set `AGENT_FLOW_REAL=1` first.

When it goes wrong

WHAT YOU SEE	WHY	FIX
The page says "Waiting for agent session" forever	The session was already open when the server started, or it runs outside the watch folder.	Open a NEW session inside the watch folder. Check the folder with <code>python start.py --status</code> and <code>config.json</code> .
Still nothing after a new session	A stale registration file for a narrower folder is catching the events (Windows).	<code>python cleanup.py</code> , then <code>python start.py --stop</code> and <code>python start.py</code> , then a new session.
"agent-flow was NOT started, to protect your Claude Code settings"	settings.json has a typing error or a byte-order mark; agent-flow would have replaced the whole file.	Error with a line number: open settings.json at that line, fix it (a comma after the last item is the usual cause), save. Byte-order mark: run <code>python install.py</code> , which removes it after a backup. Then <code>python start.py</code> .
The page never loads after logging in	The same check refused the start at logon, silently.	<code>python start.py --status</code> shows the reason; fix as in the row above.
Windows flash on every tool call	Hook copies have piled up. Before this kit, agent-flow added 1 more copy each time it started on Windows.	<code>python check_hooks.py --fix</code> , then <code>python install.py</code> to rewrite the line with forward slashes.
"Port 3001 is already in use"	Another program uses that port.	<code>python install.py --port 3002</code> , then <code>python start.py</code> , then open http://127.0.0.1:3002 .
The installer stops: Node.js not found	Node is not installed, or the terminal was opened before installing it.	Install it (see Before you start), open a NEW terminal, run again. Nothing was changed.
The installer stops: could not read settings.json	The file has a typing error in it. <code>check_hooks.py</code> shows the same fault with the line and column.	Open settings.json at that line, fix the error, run again. Your Claude Code settings were not changed.
A file called <code>setting s.json.bak-agent-flow -undo-<date></code> appeared	agent-flow rewrote your settings after starting and <code>guard.py</code> put your version back. The file is agent-flow's version, kept for you to look at.	Nothing to do. <code>logs/start.log</code> records when it happened.
Counts and costs differ from another dashboard	agent-flow's token and cost figures are estimates.	Treat them as a rough guide, not a bill.
A change you made to the screen "should work" but looks wrong	Reading the data is not the same as looking at the screen. That is how the merged-view patch failed on 2026-06-12.	Take a screenshot and look at it before calling it done.

settings.json has a comma after its last item: agent-flow is not started

```
> python start.py
agent-flow was NOT started, to protect your Claude Code settings.
C:\Users\<you>\.claude\settings.json is not valid JSON (line 111, column 4: Illegal trailing comma before end of
object). agent-flow would replace the whole file. Open it at that line (often a comma after the last item), save,
then run: python check_hooks.py
> python start.py --status
Not running for C:\Users\<you>
Last start was refused:
C:\Users\<you>\.claude\settings.json is not valid JSON (line 111, column 4: Illegal trailing comma before end of
object). agent-flow would replace the whole file. Open it at that line (often a comma after the last item), save,
then run: python check_hooks.py
> python check_hooks.py
Settings file: C:\Users\<you>\.claude\settings.json
RESULT: PROBLEM - settings.json is not valid JSON (line 111, column 4: Illegal trailing comma before end of
object).
Open the file at that line and fix it. The usual cause is a comma after the last item in a list or block. Claude
Code and agent-flow cannot read it until then, and start.py will not start agent-flow.
```

A SETTINGS FILE WITH A COMMA AFTER ITS LAST ITEM: THE START IS REFUSED, --STATUS SAYS WHY, AND CHECK_HOOKS.PY NAMES THE LINE.

WARNING

do not run `npx agent-flow-app` by hand while this kit is installed. You get 2 servers, sessions report only to the one watching the narrowest folder, and a hand-started copy runs without our settings check, so it can wipe settings.json. Use `python start.py` instead.

PIECE 1 • AGENT-FLOW

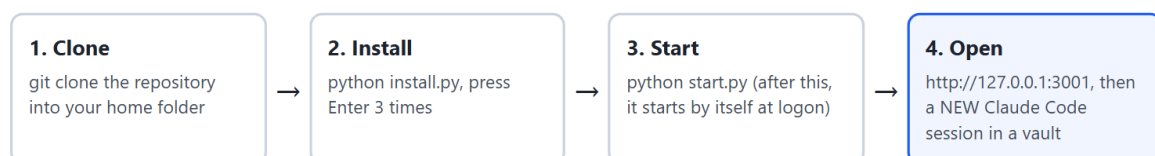
Download

The repository: <https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow>

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow; cd outliers-ws-01-agent-flow; python install.py
```

Then `python start.py` and open <http://127.0.0.1:3001>.

From download to a working screen in 4 steps



About 1 minute, plus the first download of agent-flow (up to 3 minutes on a slow connection).

THE 4 STEPS FROM DOWNLOAD TO A WORKING SCREEN.

agent-flow itself: github.com/patoles/agent-flow (Apache 2.0). Our installer, checks and guard: MIT.

DOWNLOAD THIS PIECE

<https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow>

```
git clone https://github.com/OUTLIERS-ai/outliers-ws-01-agent-flow
cd outliers-ws-01-agent-flow
python install.py
```